TP 4 - Apprentissage par renforcement

L'objectif de ce TP est de mettre en œuvre l'algorithme Q-learning de façon à créer des agents appris par renforcement pour traiter plusieurs petits jeux.

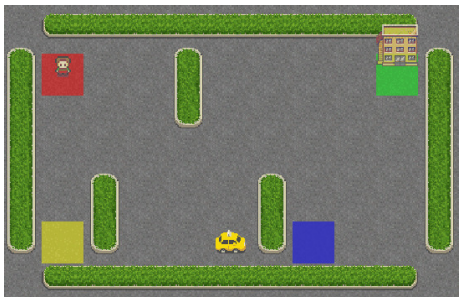
1 Contexte

L'ensemble du TP s'appuie sur la bibliothèque python Gymnasium [1], qui fournit une API pour l'apprentissage par renforcement avec la mise à disposition de plusieurs environnements de test.

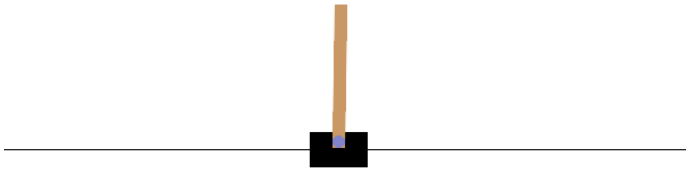
Trois applications seront étudiées (Fig. 1). Le premier jeu, *Frozen lake* [2], est un cas d'étude bien connu dans l'apprentissage renforcement, où un personnage part d'une case de départ, doit éviter des pièges (ici des trous dans la glace) et atteindre une case de destination. Quatre actions sont possibles (gauche, bas, droite, haut) et les états sont associés à une case du jeu. La récompense est de +1 si le personnage atteint la destination, 0 s'il tombe dans un trou ou n'atteint pas la sortie dans le temps imparti (100 épisodes). Deux versions du plateau sont disponibles : l'une avec 4×4 cases, l'autre avec 8×8 cases.



(a) Frozen lake



(b) Taxi



(c) Cart pole

FIGURE 1 – Aperçu des différents environnements de jeux.

Le deuxième jeu, *Taxi* [3], simule la navigation de passagers dans un environnement en grille. L'objectif est de déplacer un taxi vers la position du passager, le prendre à bord, le déplacer vers la destination souhaitée (indiquée par un immeuble) et l'y déposer. Six actions sont possibles pour le taxi : 4 directions de déplacement, le ramassage et la dépose du passager. Les états correspondent à des grilles générées aléatoirement avec plusieurs positions possibles au départ pour le taxi et le

1. `https://gymnasium.farama.org`
2. `https://gymnasium.farama.org/environments/toy_text/frozen_lake/`
3. `https://gymnasium.farama.org/environments/toy_text/taxi/`

passager, ainsi que pour la destination. La récompense est de +1 si le passager a été déposé au bon endroit et 0 si cela n'a pas été fait dans le temps imparti (200 épisodes).

Le troisième jeu, *Cart pole*[4] simule le déplacement d'un chariot le long d'un rail, portant un poteau vertical. L'objectif est de faire en sorte que ce poteau ne tombe pas, malgré son inertie lors des déplacements. Deux actions sont possibles sur le rail : gauche et droite. Les états sont décrits par quatre variables représentant la position et la vélocité du chariot d'une part, l'angle et la vélocité angulaire du poteau, d'autre part. Une récompense de +1 est donnée à chaque épisode où le poteau ne tombe pas. La simulation s'arrête dès lors que le poteau tombe ou quand la durée maximale de 500 épisodes est atteinte.

Vous pouvez tester l'exécution d'un agent aléatoire pour chacun de ces jeux avec le script play.py.

2 Implémentation du Q-learning

L'objectif de cette partie est d'activer l'option -i du script play.py pour utiliser un agent appris pour chacune des applications. Le programme fourni train.py permet d'apprendre des agents pour chaque jeu à l'aide de l'algorithme de Q-learning décrit ci-dessous.

input : α : taux d'apprentissage, ϵ : taux d'exploration, γ : taux d'escompte
output: tableau Q : Q-valeur pour chaque état s et action a
Initialiser $Q(s,a) = 0$ pour tout état s et pour tout état a;
Fixer le nombre maxmal d'épisodes max_épisode;
num_épisode $\longleftarrow$ 0;
while *num_ épisode* $<$ *max_ épisode* **do**
  $s \longleftarrow$ état initial;
  terminé $\longleftarrow$ Faux;
  while *terminé n'est pas Vrai* **do**
    `/* Choisir l'action` a `*/`
    Tirer un nombre n aléatoire entre 0 et 1;
    if $n < \epsilon$ **then**
      Choisir une action a au hasard;
    else
      $a \longleftarrow \arg\max_{a'} Q(s,a')$;
    end
    `/* Exécuter l'action` s `*/`
    $s', r,$ terminé $\longleftarrow$ exécuter(a);
    `/* Metre à jour Q(s,a) à partir de la récompense` r `et du nouvel état` s' `*/`
    $Q(s,a) \longleftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$;
    $s \longleftarrow s'$;
  end
  num_épisode $\longleftarrow$ num_épisode+1;
end

Questions

1. Pour mettre en œuvre l'algorithme ci-dessus, définissez et initialisez une structure de données pour q_values dans le constructeur de GameAgent du fichier agent.py.
2. Complétez les fonctions d'agent.py de façon à mettre en l'algorithme ci-dessus.

4. https://gymnasium.farama.org/environments/classic_control/cart_pole/

3 Test du Q-learning

Vous allez maintenant tester votre implémentation du Q-learning dans le cadre des trois applications définies au début du sujet.

Le fichier train.py permet d'apprendre les paramètres du modèle à l'aide du Q-learning et les sauvegarder dans un fichier .dump. La fonction learn est définie de façon à ce que l'apprentissage soit incrémental, en poursuivant l'apprentissage d'un modèle en vérifiant si un fichier .dump a déjà été créée pour l'application visée.

L'option -i de play.py permet quant à elle de réutiliser les modèles appris dans une exécution avec affichage.

Questions

1. Testez le Q-learning sur les applications *Frozen lake* (dans ses versions 4x4 et 8x8) et *Taxi*. Vous pourrez contrôler la qualité des modèles des agents à partir des courbes affichées à la fin de l'apprentissage par le programme train.py. Donnez le nombre d'iterations retenues pour chacun des trois modèles appris.
2. L'algorithme de Q-learning n'opère que sur des états et des actions dans des espaces discrets. Une fonction discretizeState(state) est mise à votre disposition pour l'application *Cart pole*. Modifiez train.py et test.py de façon à appeler cette fonction. Donnez le nombre d'itérations retenues pour cette dernière application.